Assignment 8: Terraform (AWS Provisioning)

Submitted by: Umair Khan

Course: DevOps — Tutedude

Date: 2 November 2026

GitHub Repository:
https://github.com/Umair-bey/tutedude-assignments/tree/main/Assignment-8

---

1. Objective

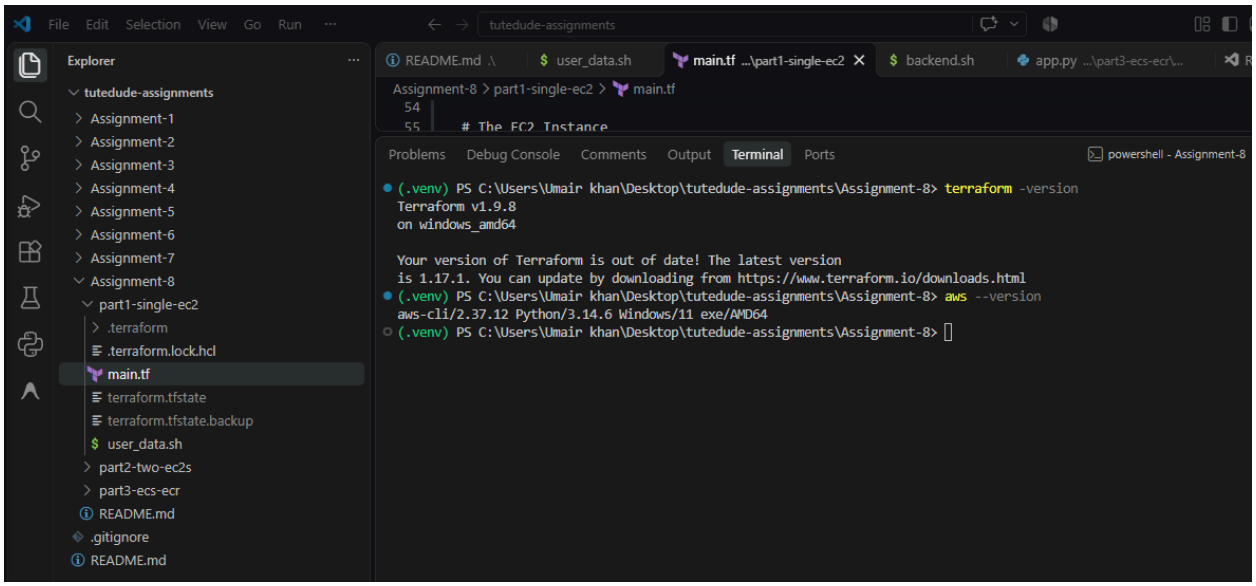
The objective of this assignment is to deploy a full-stack application consisting of a Flask backend and an Express frontend on AWS using Terraform. This involves three distinct architectural patterns:

1. Provisioning a single EC2 instance to run both applications.
2. Provisioning two separate EC2 instances inside a custom VPC, demonstrating secure internal communication.
3. Deploying the applications as Docker containers using AWS ECR, ECS Fargate, and an Application Load Balancer (ALB).

2. Prerequisites

The following tools were installed and verified on the local machine (Windows 11):

Tool	Version	Purpose
Terraform	v1.9.8	Infrastructure as Code (IaC)
AWS CLI	v2.37.12	Command-line interface for AWS
Docker Desktop	v29.8.2	Container runtime for building images
VS Code	Latest	Code editor and terminal
Git	Latest	Version control



3. Project Structure

The project source code is organized as follows:

```
None
Assignment-8/
├── part1-single-ec2/
│   ├── main.tf
│   └── user_data.sh
├── part2-two-ec2s/
│   ├── main.tf
│   ├── backend.sh
│   └── frontend.sh
├── part3-ecs-ecr/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── backend/
│   │   ├── app.py
│   │   ├── requirements.txt
│   │   └── Dockerfile
│   └── frontend/
│       ├── server.js
```

```
|       ├── package.json
|       └── Dockerfile
└── README.md
```

4. Application Architecture

The assignment demonstrates three deployment architectures:

Part 1: Single EC2 Instance
Both Flask (Port 5000) and Express (Port 3000) run on a single `t3.micro` instance. A `user_data` script installs the necessary runtimes and starts both applications on boot.

Part 2: Two Separate EC2 Instances (Custom VPC)
A custom VPC (`10.0.0.0/16`) with a public subnet is created. The Express frontend (Port 3000) communicates with the Flask backend (Port 5000) securely over the backend's Private IP.

Part 3: ECS Fargate + ECR + ALB
Docker images are pushed to AWS ECR. ECS Fargate runs the containers serverlessly. An Application Load Balancer routes internet traffic:

- `/` → Frontend ECS Task
- `/api/*` → Backend ECS Task

5. Step-by-Step Deployment

Part 1: Single EC2 Instance

Step 1: Terraform Configuration
The `main.tf` file provisions an EC2 instance and a Security Group allowing inbound traffic on ports 22, 3000, and 5000.

Step 2: User Data Script
The `user_data.sh` script runs automatically on boot to install dependencies and start both applications simultaneously in the background.

```
None
#!/bin/bash
yum update -y
```

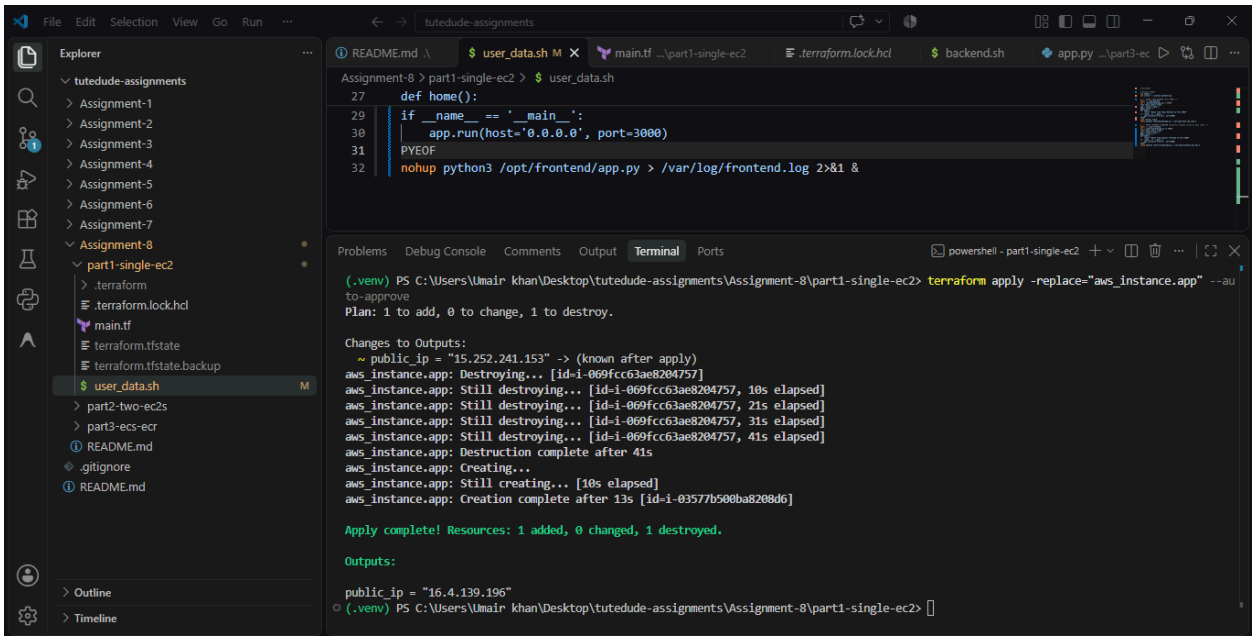
```
yum install -y python3 python3-pip

# Start Flask Backend
mkdir -p /opt/backend
cat > /opt/backend/app.py <<'PYEOF'
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home(): return "Hello from Flask Backend on Port 5000!"
if __name__ == '__main__': app.run(host='0.0.0.0', port=5000)
PYEOF
pip3 install flask
nohup python3 /opt/backend/app.py > /var/log/flask.log 2>&1 &

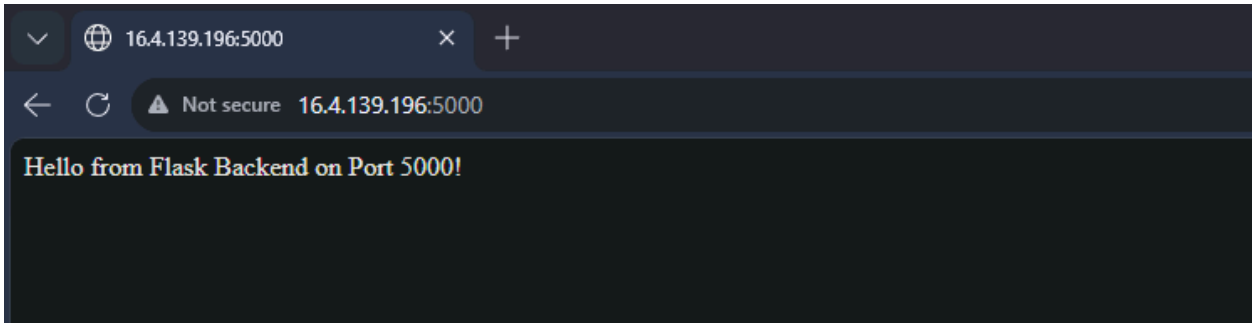
# Start "Express" Frontend (Simulated via Python for faster boot
times)
mkdir -p /opt/frontend
cat > /opt/frontend/app.py <<'PYEOF'
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home(): return "Hello from Express Frontend on Port 3000!"
if __name__ == '__main__': app.run(host='0.0.0.0', port=3000)
PYEOF
nohup python3 /opt/frontend/app.py > /var/log/frontend.log 2>&1 &
```

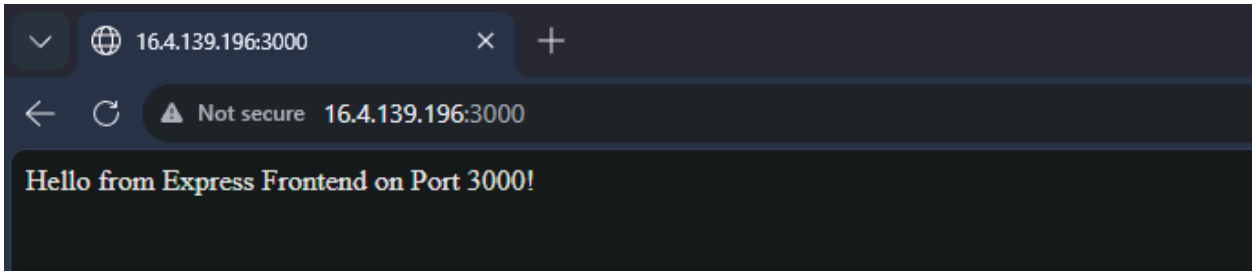
Step 3: Deploy and Verify

Run `terraform apply --auto-approve`. The output provides the `public_ip`.



After waiting for the initialization script to finish, both applications are accessible via the single instance's public IP.





Part 2: Two Separate EC2 Instances (Custom VPC)

Step 1: Terraform Networking
Terraform creates a VPC, a public subnet, an Internet Gateway, and route tables. It then provisions two EC2 instances.

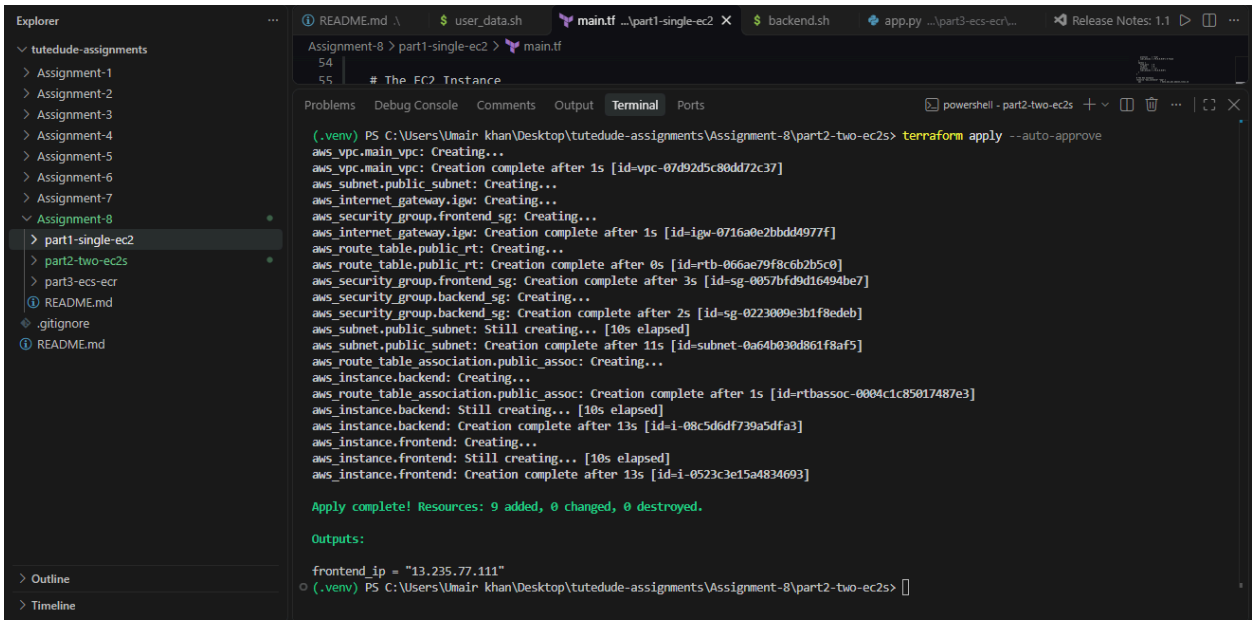
Step 2: Backend and Frontend Scripts
Two separate scripts are used:

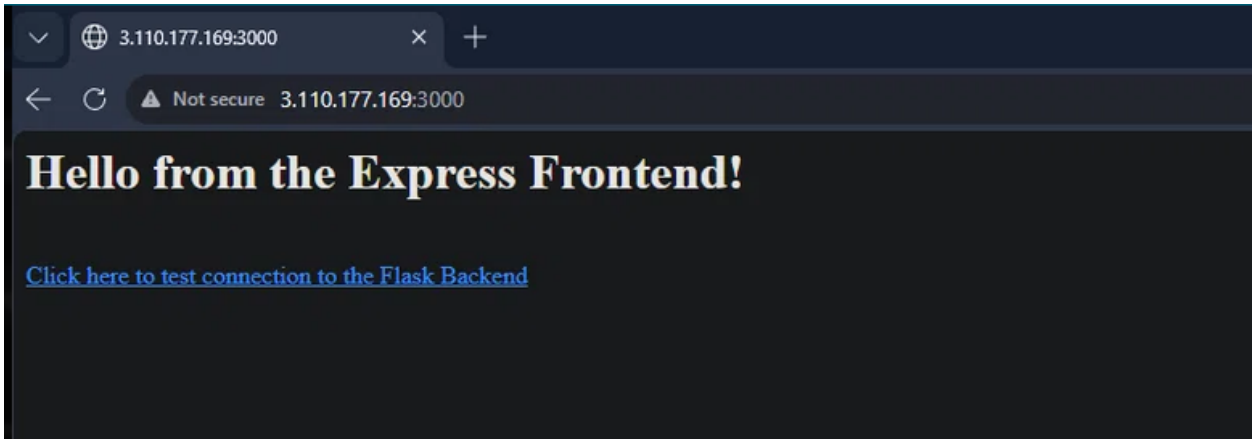
- `backend.sh`: Installs Flask and starts the backend on port 5000.
- `frontend.sh`: Installs Node.js and starts Express on port 3000.

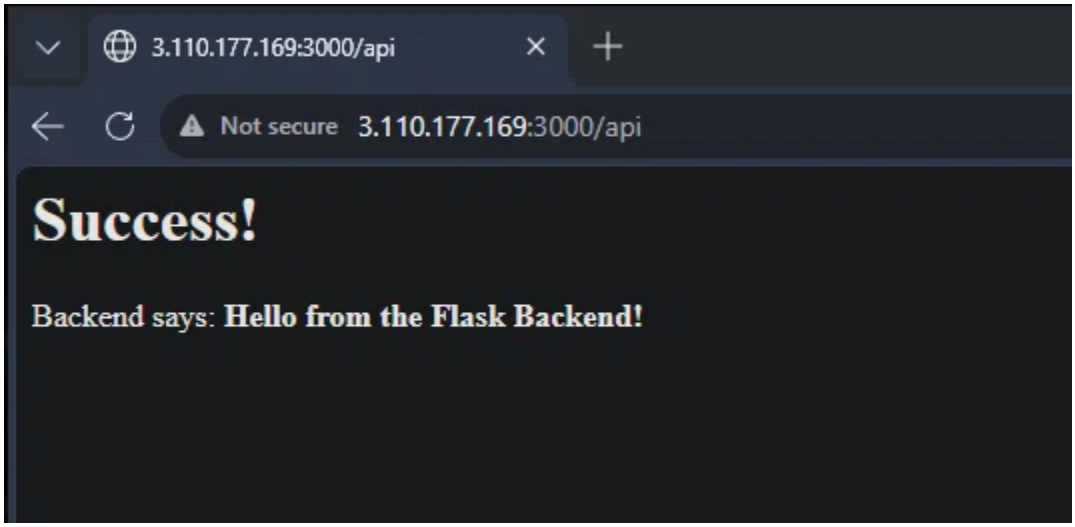
Terraform automatically injects the backend's Private IP into the frontend's script using the `replace()` function to avoid template parsing errors:

```
None
resource "aws_instance" "frontend" {
  # ...
  user_data = replace(
    file("${path.module}/frontend.sh"),
    "__REAL_BACKEND_IP__",
    aws_instance.backend.private_ip
  )
}
```

Step 3: Deploy and Verify
Run `terraform apply --auto-approve`.


Explorer
tutedude-assignments
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
part1-single-ec2
part2-two-ec2s
part3-ecs-ecr
README.md
.gitignore
README.md
Outline
Timeline
README.md
user_data.sh
main.tf ...\part1-single-ec2
backend.sh
app.py ...\part3-ecs-ecr\...
Release Notes: 1.1
Assignment-8 > part1-single-ec2 > main.tf
54
55
The EC2 Instance
Problems
Debug Console
Comments
Output
Terminal
Ports
powershell - part2-two-ec2s
(.venv) PS C:\Users\Umair khan\Desktop\tutedude-assignments\Assignment-8\part2-two-ec2s> terraform apply --auto-approve
aws_vpc.main_vpc: Creating...
aws_vpc.main_vpc: Creation complete after 1s [id=vpc-07d92d5c80dd72c37]
aws_subnet.public_subnet: Creating...
aws_internet_gateway.igw: Creating...
aws_security_group.frontend_sg: Creating...
aws_internet_gateway.igw: Creation complete after 1s [id=igw-0716a0e2bbdd4977f]
aws_route_table.public_rt: Creating...
aws_route_table.public_rt: Creation complete after 0s [id=rtb-066ae79f8c6b2b5c0]
aws_security_group.frontend_sg: Creation complete after 3s [id=sg-0057bfd9d16494be7]
aws_security_group.backend_sg: Creating...
aws_security_group.backend_sg: Creation complete after 2s [id=sg-0223009e3b1f8edeb]
aws_subnet.public_subnet: Still creating... [10s elapsed]
aws_subnet.public_subnet: Creation complete after 11s [id=subnet-0a64b030d861f8af5]
aws_route_table_association.public_assoc: Creating...
aws_instance.backend: Creating...
aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-0004c1c85017487e3]
aws_instance.backend: Still creating... [10s elapsed]
aws_instance.backend: Creation complete after 13s [id=i-08c5d6df739a5dfa3]
aws_instance.frontend: Creating...
aws_instance.frontend: Still creating... [10s elapsed]
aws_instance.frontend: Creation complete after 13s [id=i-0523c3e15a4834693]
Apply complete! Resources: 9 added, 0 changed, 0 destroyed.
Outputs:
frontend_ip = "13.235.77.111"
(.venv) PS C:\Users\Umair khan\Desktop\tutedude-assignments\Assignment-8\part2-two-ec2s>


3.110.177.169:3000
Not secure 3.110.177.169:3000
Hello from the Express Frontend!
Click here to test connection to the Flask Backend


3.110.177.169:3000/api
Not secure 3.110.177.169:3000/api
Success!
Backend says: Hello from the Flask Backend!

Part 3: ECS Fargate + ECR + ALB

Step 1: Terraform Infrastructure
Terraform provisions the ECR repositories, VPC, ALB, Target Groups, IAM roles, ECS Cluster, and Fargate Task Definitions.

Step 2: Build and Push Docker Images
Since Terraform creates empty ECR repos, we must build and push the images manually.

```
None
# Login to ECR
aws ecr get-login-password --region ap-south-1 | docker login
--username AWS --password-stdin
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com

# Build and Push Backend
cd backend
docker build -t assignment8-p3-backend .
docker tag assignment8-p3-backend:latest
"<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-backe
nd:latest"
docker push
"<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-backe
nd:latest"

# Build and Push Frontend
cd ../frontend
docker build -t assignment8-p3-frontend .
docker tag assignment8-p3-frontend:latest
"<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-front
end:latest"
docker push
"<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-front
end:latest"
```

Step 3: Force ECS Update
Because the ECS services were created before the images existed, we force a new deployment.

None

```
(.venv) PS C:\Users\Umair
khan\Desktop\tutedude-assignments\Assignment-8\part3-ecs-ecr> cd
backend
>> docker push
117647030291.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-backen
d:latest
The push refers to repository
[117647030291.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-backe
nd]
b3ec39b36ae8: Pushed
098fcf42e082: Pushed
44136fa355b3: Pushed
38513bd72563: Pushed
fc7443084902: Pushed
ea56f685404a: Pushed
9958579a52bd: Pushed
22b5728d2be9: Pushed
e688e7bd7be0: Pushed
30093bafae30: Pushed
latest: digest:
sha256:ba6c391257e208e9bc83734d7b0ef4f88c4eccb7fcdd7203ad579a8e8930
49d0 size: 856

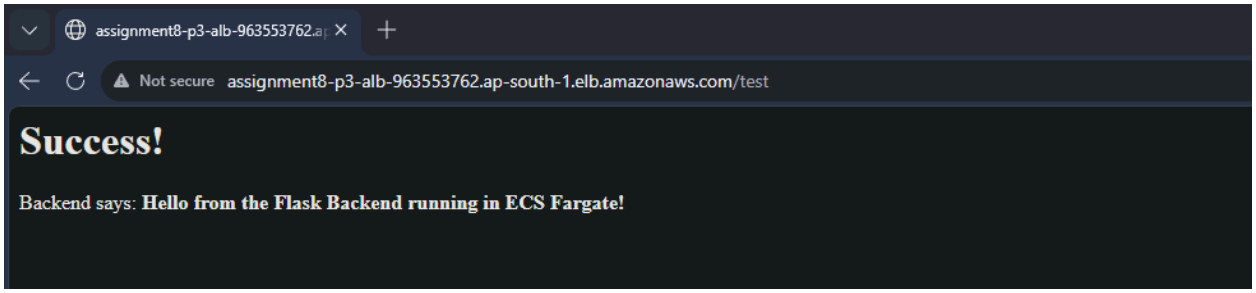
(.venv) PS C:\Users\Umair
khan\Desktop\tutedude-assignments\Assignment-8\part3-ecs-ecr> cd
frontend
>> docker push
117647030291.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-fronte
nd:latest
The push refers to repository
[117647030291.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-front
end]
eee371b9ce3f: Pushed
44136fa355b3: Pushed
7264a8db6415: Pushed
eb56c7502792: Pushed
```

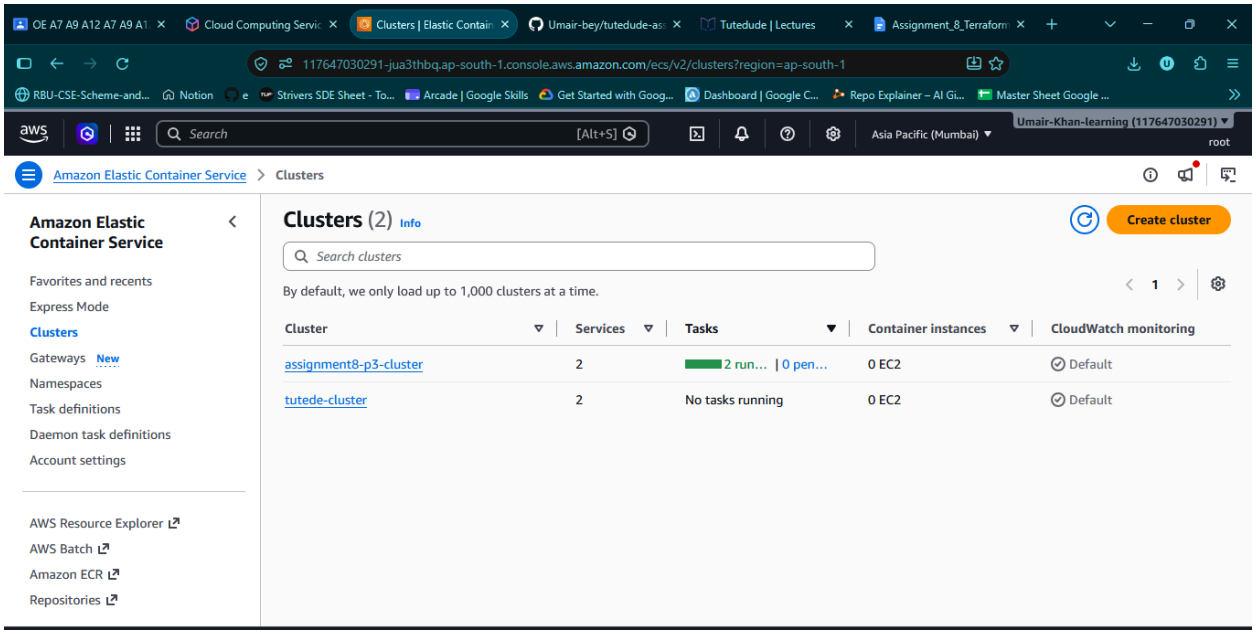
```
93b3025fe103: Pushed
d9059661ce70: Pushed
82e90382ba44: Pushed
c73684d31659: Pushed
4c4584006dd5: Pushed
e918f516d62e: Pushed
latest: digest:
sha256:87c4b92c827a0511c365ab070a82504dec0ff33e7e29a7df974573393b82
41c7 size: 856
```

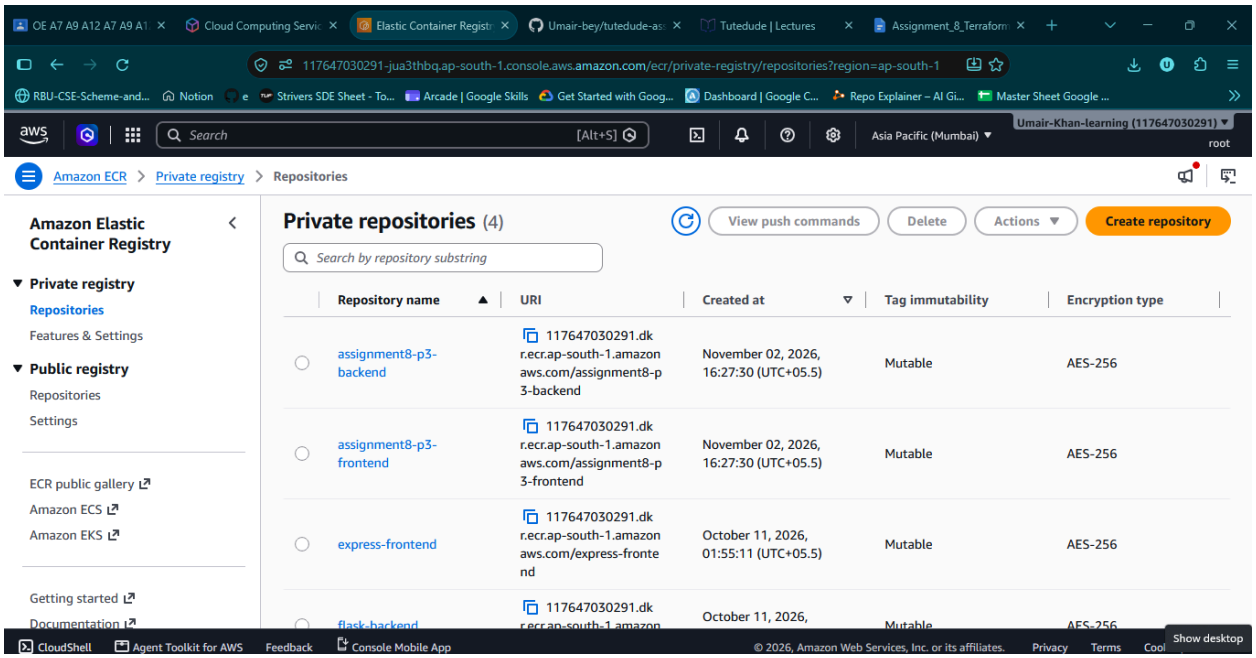
Step 4: Verify the Application

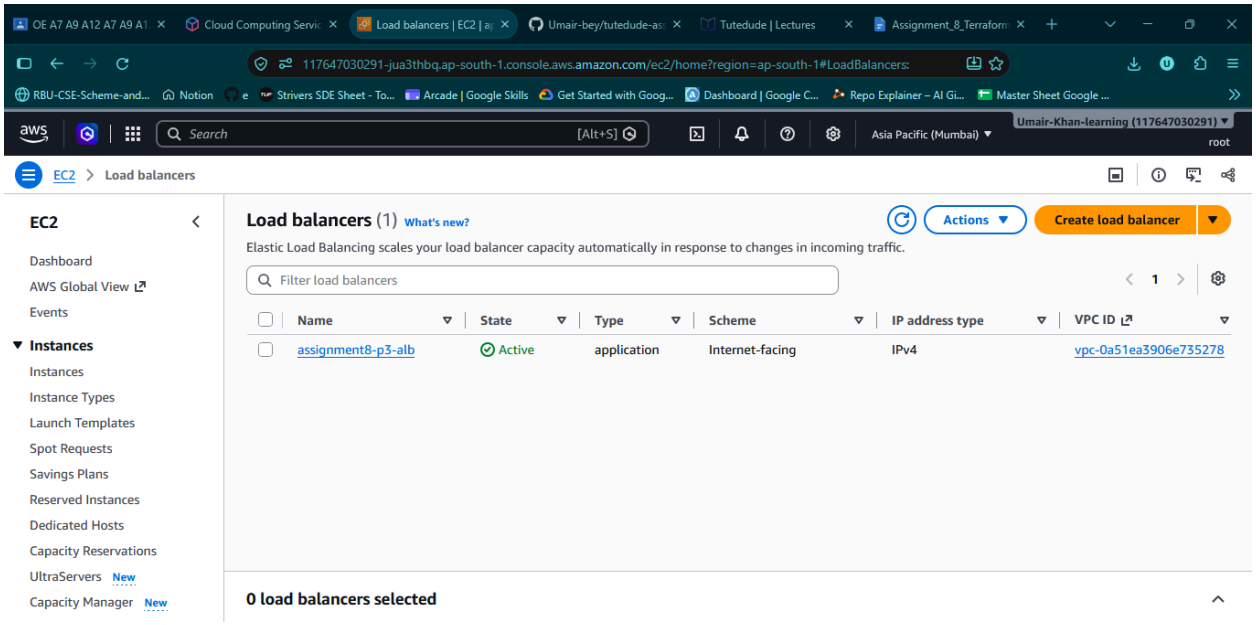
Open the ALB DNS name in the browser.






117647030291-jua3thbq.ap-south-1.console.aws.amazon.com/ecs/v2/clusters?region=ap-south-1
Search
[Alt+S]
Asia Pacific (Mumbai)
Umair-Khan-learning (117647030291)
root
Amazon Elastic Container Service
Clusters
Amazon Elastic Container Service
Favorites and recents
Express Mode
Clusters
Gateways New
Namespaces
Task definitions
Daemon task definitions
Account settings
AWS Resource Explorer
AWS Batch
Amazon ECR
Repositories
Clusters (2) Info
Search clusters
Create cluster
By default, we only load up to 1,000 clusters at a time.
Cluster
Services
Tasks
Container instances
CloudWatch monitoring
assignment8-p3-cluster
2
2 run... | 0 pen...
0 EC2
Default
tutede-cluster
2
No tasks running
0 EC2
Default


117647030291-jua3thbq.ap-south-1.console.aws.amazon.com/ecr/private-registry/repositories?region=ap-south-1
Search
[Alt+S]
Asia Pacific (Mumbai)
Umair-Khan-learning (117647030291)
root
Amazon ECR
Private registry
Repositories
Amazon Elastic Container Registry
Private registry
Repositories
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Amazon ECS
Amazon EKS
Getting started
Documentation
Private repositories (4)
View push commands
Delete
Actions
Create repository
Search by repository substring
Repository name
URI
Created at
Tag immutability
Encryption type
assignment8-p3-backend
117647030291.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-backend
November 02, 2026, 16:27:30 (UTC+05.5)
Mutable
AES-256
assignment8-p3-frontend
117647030291.dkr.ecr.ap-south-1.amazonaws.com/assignment8-p3-frontend
November 02, 2026, 16:27:30 (UTC+05.5)
Mutable
AES-256
express-frontend
117647030291.dkr.ecr.ap-south-1.amazonaws.com/express-frontend
October 11, 2026, 01:55:11 (UTC+05.5)
Mutable
AES-256
flask-backend
October 11, 2026,
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Show desktop



6. Challenges and Troubleshooting

Challenge 1: Terraform `vars map does not contain key` Error

When injecting JavaScript variables (`${...}`) into bash scripts via Terraform's `templatefile()`, Terraform attempts to evaluate them and throws an error.
Resolution: We used Terraform's `replace()` function. We placed a placeholder `__REAL_BACKEND_IP__` in the bash script and used `replace(file("..."), "__REAL_BACKEND_IP__", aws_instance.backend.private_ip)` to inject the IP safely without triggering the template engine.

Challenge 2: Docker ECR Login Fails on Windows

Docker Desktop on Windows often throws a `400 Bad Request` when piping the ECR password (`aws ecr get-login-password | docker login`).
Resolution: We ran `aws ecr get-login-password` to output the token, and then manually pasted it into the `docker login --username AWS` prompt.

Challenge 3: ECS Running Stale Code

After pushing new Docker images to ECR, ECS does not automatically detect the change if the tag is `:latest`. The container remained on the old code, causing a `404 Not Found` error.
Resolution: We forced Terraform to destroy and recreate the task definitions:

```
terraform apply -replace="aws_ecs_task_definition.backend"
-replace="aws_ecs_task_definition.frontend" --auto-approve
```

7. Key Learnings

1. **Infrastructure as Code (IaC):** Using HCL to provision, modify, and destroy AWS infrastructure predictably.
2. **Custom VPC Networking:** Configuring subnets, Internet Gateways, Route Tables, and Security Groups to control traffic flow.
3. **EC2 Bootstrapping:** Using bash scripts (`user_data`) to automate application installation on server startup.
4. **Containerization & ECR:** Building Docker images locally and securely pushing them to AWS Elastic Container Registry.
5. **ECS Fargate:** Running containers serverlessly without provisioning or managing underlying EC2 instances.
6. **Application Load Balancing:** Routing HTTP traffic to different container ports based on URL paths.
7. **State Management:** Understanding how Terraform tracks resources and how to force updates when external state (like Docker tags) changes.

8. Conclusion

This assignment demonstrated a complete Infrastructure as Code workflow using Terraform. We successfully provisioned EC2 instances, built a custom VPC, and deployed a fully serverless containerized architecture using ECS, ECR, and ALB. The applications communicate securely, traffic is balanced correctly, and the entire infrastructure can be created and destroyed with a single command. All AWS resources were destroyed via `terraform destroy` after verification to prevent unnecessary charges.

9. GitHub Repository

Full Terraform configuration, Dockerfiles, and application source code are available at:
https://github.com/Umair-bey/tutedude-assignments/tree/main/Assignment-8

10. References

- Terraform AWS Provider Docs: https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- AWS ECS Documentation: https://docs.aws.amazon.com/ecs/
- AWS ECR Documentation: https://docs.aws.amazon.com/ecr/
- Terraform Docs: https://developer.hashicorp.com/terraform/docs